

Discrete Mathematics, Week 3

Relations & Functions

JeYoung Park
SeoulTech CS&CE

2026-04-16

Part 1 — 관계(Relations)

- 관계란? 세 가지 표현 방식 (순서쌍, Boolean 행렬, 유한 그래프)
- 관계의 5가지 핵심 성질: Reflexive, Irreflexive, Symmetric, Antisymmetric, Transitive
- 관계의 결합, 합성($\circ$), 거듭제곱(R^n)
- 폐포(Closure)와 연결성 관계 R^*
- Warshall 알고리즘 — 그래프와 행렬로 동시에 보기
- 동치관계($\sim$)와 분할(Partition)
- 부분순서($\preceq$)와 Hasse diagram, Lattice, Topological Sort

Part 2 — 함수(Functions)

함수 = “존재성+유일성”을 만족하는 특수한 관계 $\rightarrow$ 단사/전사/전단사

Week 2 복습과 이번 주 연결점

지난 주에 배운 것

- **집합(Set):** 원소의 모임, $\in, \subseteq, \cup, \cap$
- **Cartesian product:**
 $A \times B = \{(a, b) \mid a \in A, b \in B\}$
- **행렬(Matrix):** 숫자의 2차원 배열
- **명제 논리:** $\wedge, \vee, \neg, \rightarrow, \forall, \exists$

이번 주에 쓸 도구

- **관계** $\subseteq A \times B$
- **Boolean 행렬:** 원소가 0 또는 1만
- 명제로 성질 정의 ($\forall a \forall b \dots$)
- 집합 연산으로 관계 결합

중심 질문

“집합의 두 원소 사이의 연결/관계를 어떻게 수학과 컴퓨터로 다룰 것인가?”

Part 1

Relations

관계 — 두 원소를 잇는 방식

관계(Relation) — 왜 배우나?

컴퓨터공학에서 관계가 나타나는 곳

- **데이터베이스**: 테이블의 행 하나 = 튜플 하나 = 관계의 원소
- **그래프**: 정점 V 위의 관계 $E \subseteq V \times V$ 가 곧 간선집합
- **의존성(Dependency)**: 파일 A 가 B 를 import $\Rightarrow (A, B) \in \text{imports}$
- **순서(Ordering)**: 빌드 순서, 작업 스케줄링의 선후 관계
- **동등성(Equality)**: 캐시 키 매칭, 집합 분할(bucket)

한 줄로

서로 다른 대상들을 “연결”하는 모든 구조는 관계로 모델링 가능하다.

관계(Relation) — 정의

Definition 1 — Binary Relation

집합 A, B에 대해 **이항관계(binary relation)** R은 $A \times B$ 의 **부분집합**이다:

$$R \subseteq A \times B$$

$(a, b) \in R$ 이면 “a는 b와 관계 R에 있다”고 하고 $a R b$ 로 쓴다.

핵심

관계는 “어떤 순서쌍들이 포함되는가”로 완전히 결정된다.
순서쌍의 집합일 뿐이다 — 그 이상도 이하도 아님.

Binary vs n-ary

이번 주는 $n = 2$ (이항관계)만 다룸. DB의 n 개 열 테이블은 n -ary relation이지만, 수학적 성질은 이항관계가 핵심.

관계 예시 — 일상과 CS

일상 예시

- A=사람 위의 “친구” 관계
- $\mathbb{Z}$ 위의 “ $\leq$ ” 관계
- $\mathbb{Z}^+$ 위의 “나눈다” 관계
- 문자열 위의 “같다(=)” 관계

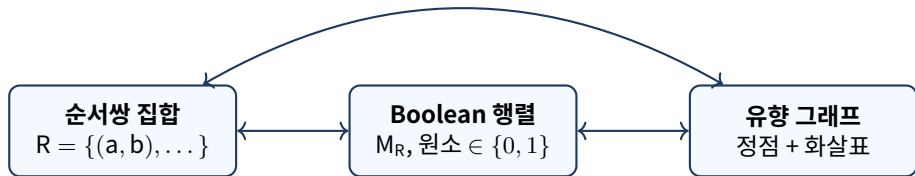
CS 예시

- 파일 A가 B를 include
- 프로세스 P가 리소스 R을 보유
- 웹페이지 p가 q를 링크
- 커밋 c_1 이 c_2 의 부모

공통점

“두 대상 사이에 연결이 있는가?”라는 yes/no 질문 — 이게 관계의 본질.

관계의 세 가지 표현 — 큰 그림



어떤 표현을 언제 쓰나?

- **순서쌍 집합**: 수학적 정의, 작은 예시 분석
- **Boolean 행렬**: 성질 판정, 빠른 계산 (bitset $O(n^2/64)$)
- **유향 그래프**: 직관적 시각화, 경로/도달성 문제

핵심

셋은 **동일한 대상**의 서로 다른 렌즈. 자유롭게 넘나들 수 있어야 함.

Boolean 행렬이란?

정의 — Boolean Matrix

원소가 오직 **0** 또는 **1**만 가능한 행렬. 일반 행렬과의 차이:

- **일반 행렬**: 원소가 실수/정수 등 자유로운 값
- **Boolean 행렬**: 원소가 $\{0, 1\}$ 뿐 — 각 칸이 “연결 있음/없음”만 표현

왜 Boolean 행렬?

관계 $R \subseteq A \times B$ 의 행렬 M_R 은 “ $(a_i, b_j) \in R$ 인가?”를 저장.
답이 **yes/no (1/0)**이므로 Boolean 행렬이 자연스러움.

주의

이번 주에 나오는 모든 관계 행렬 M_R 은 **전부 Boolean 행렬**이다 (원소가 0 또는 1만).
일반적인 덧셈/곱셈 대신 $\vee$ (OR), $\wedge$ (AND)로 연산.

세 표현의 예시 — $A = \{1, 2, 3\}$

관계: $R = \{(1, 1), (1, 2), (2, 3), (3, 1)\}$

1. 순서쌍 집합 — 위에 그대로 표기.

2. Boolean 행렬 M_R

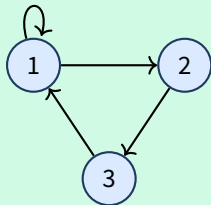
행 i = 시작점, 열 j = 도착점.

$M_R[i][j] = 1 \iff (i, j) \in R$

$$M_R = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

파란 칸: 관계에 포함되는 쌍들.

3. 유향 그래프



자기 자신으로 가는 화살표 = 자기 루프 $((1, 1))$.

예제 관계 — “나눈다(divides)”

정의

양의 정수 a, b 에 대해 $a \mid b$ (“ a 가 b 를 나눈다”)는
어떤 양의 정수 k 가 존재하여 $b = a \cdot k$
일 때. 예: $2 \mid 6$ (왜? $6 = 2 \cdot 3$), $3 \nmid 7$.

왜 이 예제?

“나눈다”는 앞으로 배울 다섯 가지 성질 중 네 가지(Reflexive, Antisymmetric, Transitive, Not Symmetric)를 한꺼번에 보여주는 좋은 예시. 또한 부분순서(Partial Order)의 대표 사례.

표기

“나눈다” 관계를 $D = \{(a, b) \mid a \mid b\}$ 라 쓰자. $(a, b) \in D \iff a \mid b$.

“나눈다” on $\{1, 2, 3, 4\}$ — 순서쌍 집합

모든 쌍을 손으로 계산

각 (a, b) 에 대해 $a \mid b$ 인지 점검:

- $1 \mid 1, 2, 3, 4$ (1은 모두 나눔) $\Rightarrow (1, 1), (1, 2), (1, 3), (1, 4)$
- $2 \mid 2, 4$ ($2 = 2 \cdot 1, 4 = 2 \cdot 2$) $\Rightarrow (2, 2), (2, 4)$
- $3 \mid 3$ 만 $\Rightarrow (3, 3)$
- $4 \mid 4$ 만 $\Rightarrow (4, 4)$

결과

$D = \{(1, 1), (1, 2), (1, 3), (1, 4), (2, 2), (2, 4), (3, 3), (4, 4)\}$

총 8개 순서쌍.

“나눈다” on $\{1, 2, 3, 4\}$ — Boolean 행렬

행렬 표현

행 i , 열 j 에서 $i \mid j$ 이면 1, 아니면 0:

$$M_D = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

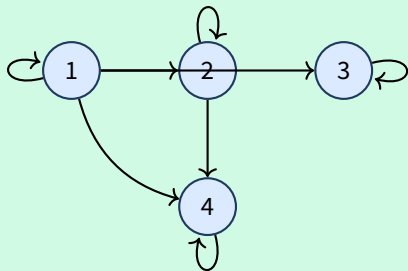
행/열 순서: 1, 2, 3, 4.

관찰할 점

- **대각선은 모두 1** — 모든 수가 자기 자신을 나눔 (앞으로 배울 Reflexive)
- **위쪽 삼각형에만 1** (대각선 제외 시) — 한 방향 관계 (Antisymmetric)
- 8개의 1 = 순서쌍 개수와 일치

“나눈다” on $\{1, 2, 3, 4\}$ — 유형 그래프

그래프 표현



읽는 법

- 각 노드에 **자기 루프** (reflexive)
- 1에서 모든 곳으로 화살표
- $2 \rightarrow 4$ 있음 ($2 \mid 4$)
- 3은 다른 곳으로 못 감

주의

화살표는 “나누는 쪽” → “나뉘는 쪽”.
방향 혼동 주의!

관계의 5가지 핵심 성질 — 큰 그림

왜 이 5가지?

관계에 어떤 성질들이 있느냐에 따라 관계는 전혀 다른 “종류”가 된다. 이 5가지 조합이 두 중요 관계(동치관계, 부분순서)를 정의함.

5가지 성질 한눈에

- **Reflexive:** “모든 원소 a 에 대해 $(a, a) \in R$ ”
- **Irreflexive:** “어떤 원소도 $(a, a) \notin R$ ”
- **Symmetric:** “ $(a, b) \in R \Rightarrow (b, a) \in R$ ”
- **Antisymmetric:** “ $(a, b) \in R \wedge (b, a) \in R \Rightarrow a = b$ ”
- **Transitive:** “ $(a, b), (b, c) \in R \Rightarrow (a, c) \in R$ ”

판정 전략

1) 정의를 정확히 / 2) 반례 하나면 성질 깨짐 / 3) 행렬로 빠르게 판정 가능.

Reflexive — 직관

한 줄 직관

모든 원소가 자기 자신과 관계를 맺고 있다.

어떤 관계가 Reflexive?

- “같다(=)”: 모든 a 에 대해 $a = a$
- “ $\leq$ ” on $\mathbb{Z}$: $a \leq a$ 는 항상 참
- “나눈다”: $a \mid a$ 이므로 reflexive
- “같은 동네 거주”: 자기 자신은 자기와 같은 동네

반례 — Reflexive 아님

“엄격한 대소” $<$: $3 < 3$ 은 거짓이므로 Not Reflexive

“친구 관계”에서 “나는 나의 친구가 아니다”로 약속한 경우

Reflexive — 정의와 행렬/그래프 판정

정의 2

$R \subseteq A \times A$ 가 **reflexive**이다 $\iff \forall a \in A, (a, a) \in R$.

행렬 판정

대각선 원소 $M_R[i][i]$ 가 전부 1이면 Reflexive.

예: M_D 에서 대각선 = (1, 1, 1, 1) $\Rightarrow$ Reflexive.

그래프 판정

모든 노드에 자기 루프가 있으면 Reflexive.

하나라도 빠지면 깨짐.

C++ 판정 (bitset)

```
for (int i=0; i<n; i++) if (!M[i][i]) return false; O(n).
```

Irreflexive — “자기 루프 하나도 없음”

정의 3

R 이 **irreflexive**이다 $\iff \forall a \in A, (a, a) \notin R$.

예시

- $<$ on $\mathbb{Z}$: $a < a$ 는 절대 거짓 $\Rightarrow$ irreflexive
- “부모 자식”: 아무도 자기 자신의 부모일 수 없음
- 자기 자신을 include하지 않는 “import”

중요 — Reflexive와 Irreflexive는 “반대”가 아님!

둘 다 아닌 경우도 가능. 예: $R = \{(1, 1), (2, 3)\}$ on $\{1, 2, 3\}$ 은

- $(2, 2) \notin R$ 이므로 Not Reflexive
- $(1, 1) \in R$ 이므로 Not Irreflexive

Reflexive의 부정 $\neq$ Irreflexive. 전자는 “모두 있다”의 부정 = “하나라도 없다”.

Symmetric — 직관

한 줄 직관

한 방향 관계가 있으면 반대 방향도 자동으로 있다 — “쌍방향”.

Symmetric한 관계

- “친구”: A가 B의 친구면 B도 A의 친구 (보통)
- “같다(=)”: $a = b \iff b = a$
- “같은 반”: 같은 반이라는 건 대칭적
- SNS의 “맞팔”: 한쪽만 follow면 symmetric 아님

Not Symmetric

“ $\leq$ ”: $3 \leq 5$ 이지만 $5 \not\leq 3$.

“나눈다”: $2 \mid 4$ 이지만 $4 \nmid 2$.

“부모-자식”: A가 B의 부모라고 B가 A의 부모는 아님.

Symmetric — 정의

정의 4

$R \subseteq A \times A$ 가 **symmetric**이다 $\iff$

$$\forall a, b \in A ((a, b) \in R \rightarrow (b, a) \in R).$$

행렬 판정 — 직관적으로 이해하기

$(a, b) \in R$ 이면 $(b, a) \in R \iff M[i][j] = M[j][i]$

$\Rightarrow$ 행렬이 주대각선 기준으로 대칭.

수식으로: $M_R = M_R^T$

판정 알고리즘

모든 (i, j) 쌍에 대해 $M[i][j] = M[j][i]$ 인지 확인. $O(n^2)$.

Symmetric 예시 — 구체적인 행렬

예시 1 — Symmetric

$R_1 = \{(1, 2), (2, 1), (1, 3), (3, 1)\}$ on $\{1, 2, 3\}$

$$M_{R_1} = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

주대각선 기준 대칭 $\Rightarrow$ Symmetric $\checkmark$

예시 2 — Not Symmetric

$R_2 = \{(1, 2), (1, 3), (3, 1)\}$ on $\{1, 2, 3\}$

$$M_{R_2} = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$$

(1, 2)는 있는데 (2, 1) 없음 — 대칭 깨짐.

그래프에서 symmetric은?

모든 간선이 **양방향** (두 개의 반대 화살표 or 무향 간선으로 그릴 수 있음).

Antisymmetric — 직관

한 줄 직관

“서로가 서로에게 관계”인 건 오직 자기 자신뿐.

비유

“A가 B의 상사이면서 B도 A의 상사” $\Rightarrow$ 이걸 $A = B$ 일 때만 가능.
즉, 두 서로 다른 원소 사이의 관계는 한 방향만 가능.

Antisymmetric한 관계

- “ $\leq$ ” on $\mathbb{Z}$: $a \leq b \wedge b \leq a \Rightarrow a = b$
- “나눈다” on $\mathbb{Z}^+$: $a \mid b \wedge b \mid a \Rightarrow a = b$
- “부모 $\rightarrow$ 자식” 관계
- “파일 포함”: 디렉토리 계층

Antisymmetric — 정의

정의 5

R 이 **antisymmetric**이다 $\iff$

$$\forall a, b \in A ((a, b) \in R \wedge (b, a) \in R \rightarrow a = b).$$

대우: $a \neq b$ 이면 (a, b) 와 (b, a) 둘 다 R 에 있을 수 없다.

행렬 판정 — 매우 중요!

모든 $i \neq j$ 에 대해: $M[i][j]$ 와 $M[j][i]$ 가 동시에 1일 수 없음.

수식으로: $i \neq j$ 면 $M[i][j] \wedge M[j][i] = 0$.

대각선은 상관없음 — $M[i][i]$ 는 뭐든 OK.

자주 하는 실수

“Antisymmetric = Not Symmetric”이 **아님**. 다음 슬라이드에서 자세히.

Symmetric vs Antisymmetric — 헷갈리지 말기

중요 — 둘은 “반대”가 아니다!

네 가지 경우가 모두 존재 가능:

	Symmetric	Not Symmetric
Antisym	$R = \{(1, 1), (2, 2)\}$ (대각선만)	$R = \{(1, 2)\}$
Not Antisym	$R = \{(1, 2), (2, 1)\}$ ($1 \neq 2$ 인데 둘 다 있음)	$R = \{(1, 2), (2, 1), (2, 3)\}$

경계 사례 — 대각선만 있는 관계

$R = \{(1, 1), (2, 2), (3, 3)\}$ on $\{1, 2, 3\}$ 는

Symmetric (모든 쌍이 $M[i][j] = M[j][i]$) 그리고 **Antisymmetric** (모든 $i \neq j$ 에 대해 둘 다 0이므로 조건 vacuously 참).

Antisymmetric 예시 — 구체적인 행렬

예시 1 — Antisymmetric

$R = \{(1, 1), (1, 2), (2, 3)\}$ on $\{1, 2, 3\}$

$$M_R = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$$

$i \neq j$ 인 쌍 $(1, 2), (2, 3)$: 각각
반대편 $((2, 1), (3, 2))$ 은 0.
 $\Rightarrow$ Antisymmetric $\checkmark$

예시 2 — Not Antisymmetric

$R = \{(1, 2), (2, 1)\}$ on $\{1, 2, 3\}$

$$M_R = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$1 \neq 2$ 인데 $(1, 2), (2, 1)$ 둘 다 있음 $\Rightarrow$
Antisymmetric 깨짐.

실전 예시 — “나눈다” 관계

M_D 에서 대각선 위쪽에만 1이 있음 (하삼각은 모두 0). 즉, $i \neq j$ 면 $M_D[i][j]$ 와 $M_D[j][i]$ 중 최대 하나만 1. $\Rightarrow$ Antisymmetric $\checkmark$.

Transitive — 직관

한 줄 직관

$a \rightarrow b$ 가 있고 $b \rightarrow c$ 가 있으면, 지름길 $a \rightarrow c$ 도 있다.

CS 비유 — 도달 가능성

네트워크에서 A에서 B로 갈 수 있고, B에서 C로 갈 수 있으면, A에서 C로도 “(결국)” 갈 수 있다. 이 “결국 갈 수 있음”을 “한 번에” 표현한 관계가 transitive.

Transitive 관계 예시

- “ $\leq$ ”: $a \leq b \wedge b \leq c \Rightarrow a \leq c$
- “나눈다”: $a \mid b \wedge b \mid c \Rightarrow a \mid c$ (왜? $b = ak_1, c = bk_2 \Rightarrow c = a(k_1k_2)$)
- “조상(ancestor)”: 조상의 조상도 조상
- “파일 include의 연쇄”

Transitive — 정의와 반례

정의 6

R 이 **transitive**이다 $\iff$

$$\forall a, b, c \in A ((a, b) \in R \wedge (b, c) \in R \rightarrow (a, c) \in R).$$

반례 — Not Transitive

“친구 관계”: A와 B가 친구, B와 C가 친구여도 A와 C는 친구 아닐 수 있음.

“부모”: A가 B의 부모, B가 C의 부모 $\Rightarrow$ A가 C의 부모? 아님 (조부모).

간단한 반례

$R = \{(1, 2), (2, 3)\}$ on $\{1, 2, 3\}$: $(1, 2), (2, 3) \in R$ 이지만 $(1, 3) \notin R \Rightarrow$ Not Transitive.

“(1, 3)을 추가하면” transitive가 됨 $\rightarrow$ Transitive Closure의 아이디어 (나중에).

Transitive 예시 — 구체적인 행렬

예시 1 — Transitive

$R = \{(1, 2), (2, 3), (1, 3)\}$ on $\{1, 2, 3\}$

$$M_R = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$$

$(1, 2), (2, 3) \in R$ 이고 $(1, 3)$ 도 있음.
다른 연쇄 없음. $\Rightarrow$ Transitive ✓

예시 2 — Not Transitive

$R = \{(1, 2), (2, 3)\}$ on $\{1, 2, 3\}$

$$M_R = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$$

$(1, 2), (2, 3) \in R$ 인데 $(1, 3) \notin R \Rightarrow$ 깨짐.

판정 알고리즘

모든 (i, j, k) 삼중에 대해 확인: $O(n^3)$. 행렬곱 이용 시 $O(n^3)$ 이지만 bitset으로 $O(n^3/64)$ 가능.

종합 예제 — 관계 $R_1, \dots, R_6$

$A = \{1, 2, 3, 4\}$ 위의 6가지 관계

- $R_1 = \{(1, 1), (1, 2), (2, 1), (2, 2), (3, 4), (4, 1), (4, 4)\}$
- $R_2 = \{(1, 1), (1, 2), (2, 1)\}$
- $R_3 = \{(1, 1), (1, 2), (1, 4), (2, 1), (2, 2), (3, 3), (4, 1), (4, 4)\}$
- $R_4 = \{(2, 1), (3, 1), (3, 2), (4, 1), (4, 2), (4, 3)\}$
- $R_5 = \{(1, 1), (1, 2), (1, 3), (1, 4), (2, 2), (2, 3), (2, 4), (3, 3), (3, 4), (4, 4)\}$
- $R_6 = \{(3, 4)\}$

할 일

각 R_i 마다 5가지 성질(Reflexive, Irreflexive, Symmetric, Antisymmetric, Transitive)을 점검. 슬라이드를 나눠 하나씩 확인.

R_1, R_2, R_3 분석

R_1

Reflexive? $(3, 3) \notin R_1 \Rightarrow$ **No**. Irreflexive? $(1, 1) \in R_1 \Rightarrow$ **No**.

Symmetric? $(3, 4) \in R_1$ 인데 $(4, 3) \notin R_1 \Rightarrow$ **No**. Antisymmetric? $(1, 2), (2, 1)$ 둘 다 있음 $\Rightarrow$ **No**. Transitive? $(3, 4), (4, 1) \in R_1$ 인데 $(3, 1) \notin R_1 \Rightarrow$ **No**.

R_2

Reflexive? $(3, 3) \notin R_2 \Rightarrow$ **No**. Irreflexive? $(1, 1) \in R_2 \Rightarrow$ **No**.

Symmetric? $(1, 2), (2, 1)$ 대칭 OK, 다른 쌍 없음 $\Rightarrow$ **Yes**. Antisymmetric? $(1, 2), (2, 1)$ 둘 다 $\Rightarrow$ **No**. Transitive? 가능한 연쇄 $(1, 2)(2, 1) \rightarrow (1, 1) \in R_2, (2, 1)(1, 2) \rightarrow (2, 2)?$
 $(2, 2) \notin R_2 \Rightarrow$ **No**.

R_3

Reflexive? $(1, 1), (2, 2), (3, 3), (4, 4)$ 모두 있음 $\Rightarrow$ **Yes**. Symmetric? $(1, 4) \in R_3$ 인데 $(4, 1) \in R_3$. 다른 쌍 점검 필요 — $(1, 2), (2, 1)$ 둘 다 있음 OK. 실은 모든 비대각 쌍이 대칭적이지 않음: 위 연산 시 R_3 는 Symmetric. Transitive 체크 필요.

R_4, R_5, R_6 분석

R_4

Reflexive? $(1, 1) \notin R_4 \Rightarrow$ **No**. Irreflexive? 대각선 원소 없음 $\Rightarrow$ **Yes**.
Symmetric? $(2, 1) \in R_4, (1, 2) \notin R_4 \Rightarrow$ **No**. Antisymmetric? 모든 $i \neq j$ 쌍 한 방향 $\Rightarrow$ **Yes**. Transitive? $(3, 2), (2, 1) \rightarrow (3, 1) \in R_4 \checkmark, (4, 2), (2, 1) \rightarrow (4, 1) \in R_4 \checkmark, (4, 3), (3, 1) \rightarrow (4, 1) \in R_4 \checkmark \Rightarrow$ **Yes**.

R_5

$R_5 = \{(i, j) : i \leq j\}$ on $\{1, 2, 3, 4\}$ (즉 " $\leq$ "). 잘 알려진 결과:
Reflexive **Yes**, Irreflexive **No**, Symmetric **No**, Antisymmetric **Yes**, Transitive **Yes**.

$R_6 = \{(3, 4)\}$

Reflexive **No**, Irreflexive **Yes**, Symmetric **No** ($(4, 3)$ 없음), Antisymmetric **Yes** (한 쌍뿐), Transitive **Yes** (연쇄 없음 $\Rightarrow$ vacuously 참).

R₁-R₆ 종합표

성질	R ₁	R ₂	R ₃	R ₄	R ₅	R ₆
Reflexive	F	F	T	F	T	F
Irreflexive	F	F	F	T	F	T
Symmetric	F	T	T	F	F	F
Antisymmetric	F	F	F	T	T	T
Transitive	F	F	T	T	T	T

관찰

- R₃: Reflexive + Symmetric + Transitive ⇒ 동치관계
- R₄, R₅: Antisymmetric + Transitive + (Reflexive or Irreflexive) ⇒ 부분순서
- R₆: Transitive(공허히)이나 Reflexive 아님

관계의 결합 — 정의

두 관계 $R_1, R_2 \subseteq A \times B$ 에 대해

관계는 **집합**이므로 집합 연산을 그대로 적용 가능:

- **합집합** $R_1 \cup R_2$: 둘 중 하나라도 관계면 포함
- **교집합** $R_1 \cap R_2$: 두 관계에 공통이면 포함
- **차집합** $R_1 - R_2$: R_1 에 있지만 R_2 에 없는 쌍들
- **대칭차** $R_1 \oplus R_2 = (R_1 \cup R_2) - (R_1 \cap R_2)$

직관

“ R_1 이나 R_2 ”라는 약한 관계를 만들 때 합집합. 둘 다 만족하는 강한 관계를 원할 때 교집합.

관계 결합 예시 — 집합 표현

$R_1 = \{(1, 1), (2, 2), (3, 3)\}$, $R_2 = \{(1, 1), (1, 2), (1, 3), (1, 4)\}$ on $\{1, 2, 3, 4\}$

R_1 : “같은 수”의 일부, R_2 : “1과 관련된 쌍”.

결합 결과

- $R_1 \cup R_2 = \{(1, 1), (1, 2), (1, 3), (1, 4), (2, 2), (3, 3)\}$
- $R_1 \cap R_2 = \{(1, 1)\}$ (둘 다 가진 것)
- $R_1 - R_2 = \{(2, 2), (3, 3)\}$ (R_1 에만 있는 것)
- $R_2 - R_1 = \{(1, 2), (1, 3), (1, 4)\}$

관계 결합 — Boolean 행렬로

Boolean 행렬 연산으로 표현 가능

원소 $\in \{0, 1\}$ 에 대해:

- 합집합 $\leftrightarrow$ 성분별 **OR** ($\vee$)
- 교집합 $\leftrightarrow$ 성분별 **AND** ($\wedge$)

수식: $M_{R_1 \cup R_2} = M_{R_1} \vee M_{R_2}$, $M_{R_1 \cap R_2} = M_{R_1} \wedge M_{R_2}$.

왜 OR/AND? — 0/1 원소로 생각

$(i, j) \in R_1 \cup R_2 \iff (i, j) \in R_1 \text{ OR } (i, j) \in R_2 \iff M_{R_1}[i][j] = 1 \text{ OR } M_{R_2}[i][j] = 1$.
Boolean 값 두 개의 OR는 성분별 $\vee$ 연산.

CS 관점

`bitset<N> u_bit = r1_bit | r2_bit;` — 64비트 단위로 한꺼번에 처리 $\Rightarrow$ 매우 빠름.

관계 결합 행렬 예시

두 Boolean 행렬

$$M_{R_1} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$M_{R_2} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

Union (성분별 OR)

$$M_{R_1} \vee M_{R_2} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Intersection (성분별 AND)

$$M_{R_1} \wedge M_{R_2} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

관계의 합성($\circ$) — 직관

한 줄 직관

“R로 한 걸음, 그리고 S로 한 걸음” — 두 단계 거쳐 간 최종 연결.

일상 비유

- R = “부모-자식” 관계: $(P, C) \in R$ 은 P 가 C 의 부모
- $R \circ R$: “부모의 부모 $\rightarrow$ 자식의 자식” = **조부모-손자** 관계
- R = “친구”: $R \circ R$ 은 **친구의 친구**

CS 비유

R = “파일 A가 B를 import”, $R \circ R$ 은 “A가 import하는 파일이 import하는 파일”. 즉, **2단계 의존성**.

관계의 합성 — 정의

Definition 7

$R \subseteq A \times B, S \subseteq B \times C$ 일 때

합성 $S \circ R \subseteq A \times C$ 는:

$$S \circ R = \{(a, c) \mid \exists b \in B, (a, b) \in R \wedge (b, c) \in S\}.$$

읽는 법

a에서 c로 “R 한 번, 그다음 S 한 번” 갈 수 있다면 $(a, c) \in S \circ R$.

중간에 어떤 b가 있든 상관없이 존재하기만 하면 됨.

순서 주의!

$S \circ R$ = “먼저 R, 그다음 S” — 함수 합성과 같은 관례.

합성 예시 — 친구의 친구

설정

$A = \{\text{Alice, Bob, Carol, Dave}\}$, $R = \text{“친구”}$

- (Alice, Bob), (Alice, Carol)
- (Bob, Alice), (Bob, Dave)
- (Carol, Alice)
- (Dave, Bob)

$R \circ R = \text{친구의 친구 (자기 자신 포함 가능)}$

각 (a, c) 가 $R \circ R$ 에 있는지 = 어떤 b 가 존재하여 $a-b-c$ 인지.

예: (Alice, Dave)? Alice의 친구 중 Dave의 친구가 있나? Alice-Bob-Dave ✓

(Alice, Alice)? Alice-Bob-Alice ✓ (Bob이 양쪽 다 친구)

$R \circ R = \{(\text{Alice, Alice}), (\text{Alice, Dave}), (\text{Bob, Bob}), (\text{Bob, Carol}), (\text{Carol, Bob}), (\text{Carol, Carol}), (\text{Dave, Alice}), (\text{Dave, Dave})\}$

Boolean 행렬곱 $\odot$ — 정의

기호와 정의

일반 행렬곱: $(AB)_{ij} = \sum_k A_{ik}B_{kj}$ (덧셈 + 곱셈)

Boolean 행렬곱 $\odot$: 덧셈을 OR, 곱셈을 AND로 대체:

$$(A \odot B)_{ij} = \bigvee_{k=1}^n (A_{ik} \wedge B_{kj}).$$

왜 OR/AND?

일반 행렬곱은 “가능한 모든 경로의 수”를 셈. Boolean에서는 “1번이라도 있으면 1” $\Rightarrow$ 존재 여부만 체크 $\Rightarrow$ OR.

각 k 항 $A_{ik} \wedge B_{kj} = 1 \iff i \rightarrow k$ 경로 있고 $k \rightarrow j$ 경로 있음.

⊙ 계산 예시 — 2×2

예시

$$A = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}. \quad A \odot B = ?$$

한 칸씩 계산

각 $(A \odot B)_{ij} = \bigvee_k (A_{ik} \wedge B_{kj})$:

- $(1, 1)$: $(A_{11} \wedge B_{11}) \vee (A_{12} \wedge B_{21}) = (1 \wedge 0) \vee (0 \wedge 1) = 0 \vee 0 = \mathbf{0}$
- $(1, 2)$: $(A_{11} \wedge B_{12}) \vee (A_{12} \wedge B_{22}) = (1 \wedge 1) \vee (0 \wedge 1) = 1 \vee 0 = \mathbf{1}$
- $(2, 1)$: $(A_{21} \wedge B_{11}) \vee (A_{22} \wedge B_{21}) = (1 \wedge 0) \vee (1 \wedge 1) = 0 \vee 1 = \mathbf{1}$
- $(2, 2)$: $(A_{21} \wedge B_{12}) \vee (A_{22} \wedge B_{22}) = (1 \wedge 1) \vee (1 \wedge 1) = 1 \vee 1 = \mathbf{1}$

결과

$$A \odot B = \begin{pmatrix} 0 & 1 \\ 1 & 1 \end{pmatrix}$$

⊙ 계산 예시 — 3×3 (한 칸 자세히)

예시

$$A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 1 & 0 \end{pmatrix}, \quad B = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

$(A \odot B)_{13}$ 자세히

$$\begin{aligned} (A \odot B)_{13} &= (A_{11} \wedge B_{13}) \vee (A_{12} \wedge B_{23}) \vee (A_{13} \wedge B_{33}) \\ &= (1 \wedge 1) \vee (0 \wedge 1) \vee (1 \wedge 0) = 1 \vee 0 \vee 0 = \mathbf{1} \end{aligned}$$

전체 결과

$$A \odot B = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{pmatrix} \quad (\text{나머지 8개 칸도 같은 방식})$$

합성 = Boolean 행렬곱

핵심 연결 — Theorem

$R \subseteq A \times B, S \subseteq B \times C$ 이면

$$M_{S \circ R} = M_R \odot M_S$$

증명 (짧게)

$$(M_R \odot M_S)_{ij} = \bigvee_k (M_R)_{ik} \wedge (M_S)_{kj} = 1$$

$$\iff \text{어떤 } k \text{에 대해 } (a_i, b_k) \in R \wedge (b_k, c_j) \in S$$

$$\iff (a_i, c_j) \in S \circ R \iff (M_{S \circ R})_{ij} = 1.$$



실용적 의미

합성이 행렬곱으로 바뀌니 컴퓨터로 아주 쉽게 계산. bitset으로 더 빠르게: $O(n^3/64)$.

관계의 거듭제곱 R^n — 정의

정의 8

$R \subseteq A \times A$ 에 대해

- $R^1 = R$
- $R^{n+1} = R \circ R^n$ (n 번 가고 한 번 더)

행렬로: $M_{R^n} = M_R \odot M_R \odot \cdots \odot M_R$ (n 번).

직관

$(a, b) \in R^n \iff a$ 에서 b 로 정확히 n 걸음에 가능.

예: $R = \text{“친구”} \Rightarrow R^3 = \text{“친구의 친구의 친구”}$.

계산 예시

$R = \{(1, 2), (2, 3), (3, 1)\}$ on $\{1, 2, 3\}$:

$R^2 = \{(1, 3), (2, 1), (3, 2)\}$, $R^3 = \{(1, 1), (2, 2), (3, 3)\}$.

Theorem 1 — Transitive 판정

Theorem 1

$R \circ R$ transitive $\iff \forall n \geq 1, R^n \subseteq R$.

증명 ($\Leftarrow$)

$\forall n, R^n \subseteq R$ 이라 가정. 특히 $R^2 \subseteq R$.

$(a, b), (b, c) \in R$ 이면 $(a, c) \in R \circ R = R^2 \subseteq R. \therefore$ transitive. □

증명 ($\Rightarrow$) — 수학적 귀납법

Base ($n = 1$): $R^1 = R \subseteq R$.

Step: $R^n \subseteq R$ 이라 가정. $(a, c) \in R^{n+1}$ 이면, 어떤 b 에 대해 $(a, b) \in R, (b, c) \in R^n \subseteq R$.
Transitive이므로 $(a, c) \in R$. 따라서 $R^{n+1} \subseteq R$. □

폐포(Closure) — 직관

한 줄 직관

주어진 관계에 **최소한만** 더 집어넣어서, 원하는 성질을 만족시키는 관계.

비유 — 방 정리

“이 방을 깨끗이 정리하라” = 원래 물건들은 그대로, **꼭 필요한 정리만** 한다는 느낌. 더하지 말고 덜하지도 말고 최소한만.

Closure는 관계에 “필요한 쌍들만 추가”해서 목표 성질(reflexive/symmetric/transitive)을 달성.

왜 “최소”가 중요한가?

그냥 “모든 쌍”을 넣으면 항상 모든 성질 만족. 그건 의미 없음. **원래 관계가 담고 있는 정보를 유지**하면서 딱 필요한 것만 추가해야 유용함.

Closure — 정의

Definition 9 — Closure

관계 $R \subseteq A \times A$ 가 어떤 성질 P 를 안 가질 때, P -폐포는

- 1 R 을 포함하고 ($R \subseteq S$),
- 2 성질 P 를 만족하며,
- 3 이 두 조건을 만족하는 가장 작은 관계 S .

세 종류

- **Reflexive closure:** $r(R) = R \cup \Delta$, $\Delta = \{(a, a) : a \in A\}$ (대각선 추가)
- **Symmetric closure:** $s(R) = R \cup R^{-1}$ ($R^{-1} = \{(b, a) : (a, b) \in R\}$)
- **Transitive closure:** $t(R) = R^*$ (이후 설명)

Reflexive Closure — 예시

$R = \{(1, 2), (2, 3), (1, 3)\}$ on $\{1, 2, 3\}$

Reflexive인가? $(1, 1), (2, 2), (3, 3)$ 없음 $\Rightarrow$ **No**.

Reflexive Closure

빠진 대각선 원소 추가: $(1, 1), (2, 2), (3, 3)$.

$r(R) = R \cup \{(1, 1), (2, 2), (3, 3)\} = \{(1, 1), (1, 2), (1, 3), (2, 2), (2, 3), (3, 3)\}$.

행렬 관점

$M_{r(R)} = M_R \vee I$ (단위 행렬과 성분별 OR).

$(1, 1), (2, 2), \dots$ 자리에 1 강제로 채움.

Symmetric Closure — 예시

$$R = \{(1, 2), (2, 3), (1, 3)\}$$

Symmetric? (1, 2) 있는데 (2, 1) 없음 $\Rightarrow$ **No**.

Symmetric Closure

$$R^{-1} = \{(2, 1), (3, 2), (3, 1)\}.$$

$$s(R) = R \cup R^{-1} = \{(1, 2), (2, 1), (2, 3), (3, 2), (1, 3), (3, 1)\}.$$

행렬 관점

$$M_{s(R)} = M_R \vee M_R^T \text{ (원본과 전치의 OR).}$$

경로(Path)와 연결성 관계 R^*

정의 — Path

관계 R 의 그래프에서 a 에서 b 로 가는 **경로 (길이 n)**는:

$$(a, x_1), (x_1, x_2), \dots, (x_{n-1}, b) \in R.$$

즉, $a \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow b$ 같은 n 개 간선 연쇄.

정의 — Connectivity Relation R^*

$a \rightarrow b$ 로 가는 **어떤 길이든 경로가 존재하면** $(a, b) \in R^*$:

$$R^* = \bigcup_{n=1}^{\infty} R^n.$$

핵심 사실

$R^* = R$ 의 **transitive closure**. 즉, $t(R) = R^*$.

Lemma 1 — 경로 길이 n 이하로 충분

Lemma 1

$|A| = n$ 이고 $a, b \in A, a \neq b$. a 에서 b 로 가는 경로가 있다면, 길이 $\leq n - 1$ 인 경로도 있다.
($a = b$ 인 순환 경로라면 길이 $\leq n$ 만으로 충분.)

직관 — 비둘기집 원리

n 개 정점만 있는데 경로가 n 보다 길면, 같은 정점을 두 번 이상 방문. 그 사이 구간(loop)을 잘라내면 더 짧은 경로 가능.

증명

경로 $a = x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_k = b, k > n - 1$ 이라 하자. 정점은 n 개뿐이므로 어떤 $i < j$ 에 대해 $x_i = x_j$. $x_0, \dots, x_i, x_{j+1}, \dots, x_k$ 로 자르면 더 짧은 경로. 이를 반복하면 길이 $\leq n - 1$ 경로 획득. $\square$

Theorem 2 — 유한함으로 충분

Theorem 2

$|A| = n$. 그러면

$$R^* = R \cup R^2 \cup R^3 \cup \dots \cup R^n.$$

증명

($\supseteq$) 명백 (모든 R^k 는 R^* 에 포함).

($\subseteq$) $(a, b) \in R^*$ 이면 어떤 $k \geq 1$ 에 대해 $(a, b) \in R^k$. Lemma 1에 의해 $k \leq n$ 인 어떤 k 에 대해서도 $(a, b) \in R^k$. $\therefore R^* \subseteq \bigcup_{k=1}^n R^k$. □

실용적 의미

Transitive closure를 계산할 때 **무한히 많은 거듭제곱**이 아니라 **n 번만** 계산하면 충분. 하지만 n 번 계산도 $O(n^4)$ — 비싸다. $\Rightarrow$ **Warshall 등장**.

Warshall 알고리즘 — 동기

Naive 방법 — 왜 비효율적?

Theorem 2: $R^* = R \cup R^2 \cup \dots \cup R^n$.

계산: Boolean 행렬곱 $n - 1$ 번, 각 곱셈 $O(n^3) \Rightarrow$ 총 $O(n^4)$.

너무 느림. n 이 1000이면 10^{12} 번 연산.

Warshall의 아이디어

거듭제곱을 안 하고, 대신 “중간 정점 집합”을 하나씩 허용해가며 누적. $O(n^3)$.

핵심 통찰 — “ k 번 노드를 거쳐도 되는가?”

$W_k[i][j] = 1 \iff$ 중간 정점으로 $\{1, 2, \dots, k\}$ 만 사용하여 $i \rightarrow j$ 경로가 있다.

W_0 : 중간 정점 안 씀 = 직접 간선만. W_n : 모든 정점 허용 = transitive closure.

핵심 점화식

$$W_k[i][j] = W_{k-1}[i][j] \vee (W_{k-1}[i][k] \wedge W_{k-1}[k][j]).$$

왜 이 식?

“중간 $\{1, \dots, k\}$ 사용 경로”는 두 경우로 분해:

- k 번을 안 거침 $\Rightarrow \{1, \dots, k-1\}$ 만으로 이미 있음 $\Rightarrow W_{k-1}[i][j]$
- k 번을 거침 $\Rightarrow i$ 에서 k 까지 $\{1, \dots, k-1\}$ 로, 그리고 k 에서 j 까지 $\{1, \dots, k-1\}$ 로
 $\Rightarrow W_{k-1}[i][k] \wedge W_{k-1}[k][j]$

둘 중 하나라도 가능하면 $W_k[i][j] = 1$.

Warshall — 의사코드

Pseudocode

```
procedure Warshall(M: n x n Boolean matrix)
  W := M
  for k := 1 to n
    for i := 1 to n
      for j := 1 to n
        W[i][j] := W[i][j] OR (W[i][k] AND W[k][j])
  return W // W = transitive closure
```

복잡도

3중 for loop $\Rightarrow O(n^3)$. **bitset** 구현 시 $O(n^3/64)$.

In-place 업데이트 OK

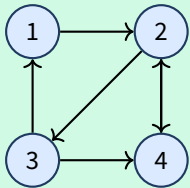
k번째 iteration에서 $W[i][k]$ 나 $W[k][j]$ 가 이미 업데이트되어도 결과는 동일 (증명: $W[k][k]$ 는 k번 iteration 전후 안 바뀜).

Warshall 예제 — 초기 그래프 W_0

예제 그래프 ($A = \{1, 2, 3, 4\}$)

초기 간선: (1, 2), (2, 3), (2, 4), (3, 1), (3, 4), (4, 2)

그래프 W_0



행렬 W_0

$$W_0 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

(행/열: 1, 2, 3, 4)

중간 정점 없이 직접 가는 간선만.

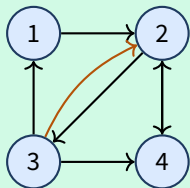
W_1 — 정점 1을 중간에 허용

무엇을 체크? — $i \rightarrow 1 \rightarrow j$ 경로

1로 들어오는 간선: $3 \rightarrow 1$. 1에서 나가는 간선: $1 \rightarrow 2$.

⇒ 새로 추가: $(3, 1) + (1, 2) = (3, 2)$.

그래프 W_1 (주황=새 간선)



행렬 W_1

$$W_1 = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

노랑: 새로 1이 된 칸.
 $(3, 2)$ 가 추가됨.

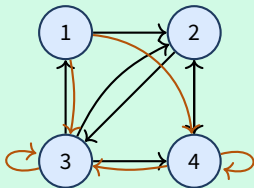
W_2 — 정점 $\{1, 2\}$ 를 중간에 허용

무엇을 체크? — $i \rightarrow 2 \rightarrow j$ 경로 (W_1 사용)

2로 들어오는: 1, 3 (새), 4. 2에서 나가는: 3, 4.

새 간선: (1, 3), (1, 4), (3, 3), (4, 3), (4, 4) — 5개 추가.

그래프 W_2



행렬 W_2

$$W_2 = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \end{pmatrix}$$

노랑 5칸 추가됨.

W_3 — 정점 $\{1, 2, 3\}$ 를 중간에 허용

무엇을 체크? — $i \rightarrow 3 \rightarrow j$ 경로 (W_2 사용)

3으로 들어오는: 1(새), 2, 3(self, 새), 4(새). 3에서 나가는: 1, 2, 3(self, 새), 4.
새 간선: (1, 1), (2, 1), (2, 2), (4, 1) — 4개 추가.

그래프 변화

이제 거의 모든 쌍이 연결됨.

새로 추가된 간선 (주황):

(1, 1): 1의 자기 루프

(2, 1), (4, 1): 1로 가는 길 생김

(2, 2): 2의 자기 루프

그래프는 이제 “거의 완전 강연결”에 가까움.

행렬 W_3

$$W_3 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

16칸 중 15칸이 1.

W_4 — 정점 $\{1, 2, 3, 4\}$ 모두 허용 (최종)

무엇을 체크? — $i \rightarrow 4 \rightarrow j$ 경로 (W_3 사용)

4로 들어오는: 1(새), 2, 3, 4(self, 새). 4에서 나가는: 1(새), 2, 3(새).
이미 모든 조합이 W_3 에 있음 — 새 간선 없음!

최종 행렬 $W_4 = R^*$

$$W_4 = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

완전행렬 = 모든 쌍이 도달 가능.

결론

원래 그래프는 **strongly connected** (모든 i 에서 모든 j 로 도달 가능)였음을 확인.

Warshall 4번의 iteration으로 $O(n^3)$ 에 판정 완료.

Warshall 정확성 — invariant로 증명

주장

각 k 에 대해, $W_k[i][j] = 1 \iff$ 중간 정점으로 $\{1, \dots, k\}$ 만 사용하여 $i \rightarrow j$ 경로가 존재.

수학적 귀납법

Base ($k = 0$): 중간 정점 없이 = 직접 간선 = M_R . $\checkmark$

Step: W_{k-1} 이 정확하다고 가정. $i \rightarrow j$ 경로가 $\{1, \dots, k\}$ 만 거친다면 두 경우:

- 경로에 k 가 없음: $\{1, \dots, k-1\}$ 만으로 충분 $\Rightarrow W_{k-1}[i][j] = 1$
- 경로에 k 가 있음: $i \rightarrow \dots \rightarrow k \rightarrow \dots \rightarrow j$. 두 조각 모두 $\{1, \dots, k-1\}$ 로 충분 $\Rightarrow W_{k-1}[i][k] = W_{k-1}[k][j] = 1$

둘 중 하나 참 $\iff W_{k-1}[i][j] \vee (W_{k-1}[i][k] \wedge W_{k-1}[k][j]) = W_k[i][j]$. □

동치관계(Equivalence Relation)

같은 부류끼리 묶기

동치관계 — 직관

한 줄 직관

“같은 부류로 묶기”의 일반화. 원소들을 겹치지 않는 그룹으로 나눌 때 사용.

일상 예시

- “같은 반에 있음” — 학생들을 반별로 묶음
- “같은 생일월” — 12개 그룹
- “인종/나이 등 기준이 일치” — 분류

CS 예시

- 해시맵에서 “같은 해시값” $\Rightarrow$ 같은 버킷
- 그래프에서 “같은 연결 성분”
- Git branch의 “같은 조상 커밋”
- 모듈러 산술: “7로 나눈 나머지가 같음”

동치관계 — 정의

Definition 10

관계 $R \subseteq A \times A$ 가 동치관계(equivalence relation)이다 $\iff$

- **Reflexive:** $\forall a, (a, a) \in R$
- **Symmetric:** $(a, b) \in R \Rightarrow (b, a) \in R$
- **Transitive:** $(a, b), (b, c) \in R \Rightarrow (a, c) \in R$

왜 이 세 성질?

“같다”가 가져야 할 최소한의 조건:

- 자기 자신과는 항상 같다 (reflexive)
- “a가 b와 같다”와 “b가 a와 같다”는 같은 말 (symmetric)
- $a = b$ 이고 $b = c$ 면 $a = c$ (transitive)

표기

R 이 동치관계면 aRb 대신 $a \sim b$ 로 쓰는 게 표준. 읽기: “a와 b는 동치”.

동치관계 예시

예시 1 — 같다(=)

반사, 대칭, 추이 모두 만족. 동치관계 중 가장 작은 것 (모든 원소가 자기 자신과만 관계).

예시 2 — 같은 길이의 문자열

$A = \text{영어 단어 집합}$. $a \sim b \iff a \text{와 } b \text{의 글자 수가 같다.}$

“cat”, “dog”, “cab”은 모두 동치 부류에 함께.

예시 3 — 같은 연결 성분 (그래프)

무향 그래프에서 “ u 와 v 가 같은 연결 성분에 있다”는 동치관계.

Union-Find 자료구조의 정확성은 이 동치관계에 기반.

중요한 동치관계 — 합동($\equiv \pmod{m}$)

정의

양의 정수 m 에 대해 “ m 을 법으로 하는 합동”은

$$a \equiv b \pmod{m} \iff m \mid (a - b).$$

즉, a, b 를 m 으로 나눈 나머지가 같음.

예시 ($m = 7$)

$10 \equiv 3 \pmod{7}$ (차 7은 7로 나누어짐). $15 \equiv 1 \pmod{7}$.

나머지 0, 1, 2, 3, 4, 5, 6에 따라 정수들을 7개 그룹으로 나눌 수 있음.

왜 중요? — CS 활용

- 해시 함수: $\text{hash}(x) \% m$
- 순환 버퍼, 에라토스테네스의 체
- 암호학 (RSA 등)
- 체크섬

합동은 동치관계 — 증명

주장

$\equiv \pmod{m}$ 은 $\mathbb{Z}$ 위의 동치관계.

증명

Reflexive: $a - a = 0, m \mid 0$ ✓.

Symmetric: $m \mid (a - b) \Rightarrow a - b = mk \Rightarrow b - a = m(-k) \Rightarrow m \mid (b - a)$. ✓

Transitive: $m \mid (a - b)$ 이고 $m \mid (b - c)$ 라 하자.

$a - b = mk_1, b - c = mk_2 \Rightarrow a - c = (a - b) + (b - c) = m(k_1 + k_2)$.

따라서 $m \mid (a - c)$. ✓

세 성질 모두 만족 $\Rightarrow$ 동치관계.



동치부류(Equivalence Class)

정의 11

동치관계 $\sim$ 에서 원소 a 의 동치부류는

$$[a] = \{x \in A : x \sim a\}.$$

즉, a 와 동치인 모든 원소의 집합.

예시 — $(\text{mod } 3)$ 동치관계 on $\mathbb{Z}$

- $[0] = \{\dots, -6, -3, 0, 3, 6, 9, \dots\}$ (3의 배수)
- $[1] = \{\dots, -5, -2, 1, 4, 7, 10, \dots\}$ (3으로 나눈 나머지 1)
- $[2] = \{\dots, -4, -1, 2, 5, 8, 11, \dots\}$ (나머지 2)

모든 정수가 이 세 그룹 중 정확히 하나에 속함.

관찰

$[0] = [3] = [6] = \dots$ — 동치관계인 원소끼리는 같은 부류.

분할(Partition) — 직관

한 줄 직관

집합을 “겹치지 않고 빠짐없이” 나누는 것.

비유 — 퍼즐 조각

한 그림을 퍼즐로 나누면:

- 모든 조각을 합치면 원래 그림 (덮음)
- 조각끼리 겹치지 않음 (서로소)

집합 분할도 정확히 같은 조건.

정의 12

집합 A 의 **분할**은 부분집합들의 집합 $\{A_1, A_2, \dots\}$ 로서

- 1 각 $A_i \neq \emptyset$
- 2 서로소: $i \neq j$ 이면 $A_i \cap A_j = \emptyset$
- 3 덮음: $\bigcup_i A_i = A$

동치관계 $\leftrightarrow$ 분할 — Theorem

Theorem 3

집합 A 위의 **동치관계**와 A 의 **분할** 사이에 일대일 대응이 있다.

핵심 아이디어

- 동치관계 $\sim \Rightarrow$ 동치부류들이 A 의 분할을 이룸
- 분할 $\{A_1, A_2, \dots\} \Rightarrow$ “같은 블록 안에 있음” 관계는 동치관계

실용적 의미

“원소들을 그룹으로 나눈다”와 “원소들 사이의 동등 관계”는 완전히 같은 개념.
 $\Rightarrow$ Union-Find로 분할 관리하면 동치관계 쿼리를 해결.

Theorem 3 증명 (동치관계 $\rightarrow$ 분할)

보일 것

동치부류들 $\{[a] : a \in A\}$ 이 A 의 분할을 이룸.

증명

1. 비어있지 않음: $a \in [a]$ (reflexive). $\checkmark$
2. 덮음: 임의의 $a \in A$ 에 대해 $a \in [a]$, 따라서 $A = \bigcup_a [a]$. $\checkmark$
3. 서로소: 두 부류 $[a], [b]$ 를 보자.
 - $[a] \cap [b] \neq \emptyset$ 이면 어떤 c 가 $c \sim a$ 이고 $c \sim b$.
 - Symmetric으로 $a \sim c$, 그리고 $c \sim b \Rightarrow$ transitive로 $a \sim b$.
 - 임의의 $x \in [a]$ 에 대해 $x \sim a \sim b \Rightarrow x \in [b]$. 반대도 성립.
 - 따라서 $[a] = [b]$.

즉 두 부류는 같거나 서로소.



Theorem 3 증명 (분할 $\rightarrow$ 동치관계)

보일 것

분할 $\{A_1, A_2, \dots\}$ 에서 " $x \sim y \iff$ 같은 A_i 에 속함"은 동치관계.

증명

Reflexive: 모든 x 는 어떤 A_i 에 속함 $\Rightarrow x \sim x$. $\checkmark$

Symmetric: x, y 가 같은 A_i 에 속하면 y, x 도 같은 A_i 에 속함. $\checkmark$

Transitive: $x \sim y$ 이고 $y \sim z$ 라 하자.

- $x, y \in A_i$ (어떤 i), $y, z \in A_j$ (어떤 j)
- $y \in A_i \cap A_j \Rightarrow$ 서로소 조건에 의해 $i = j$
- 따라서 $x, z \in A_i, x \sim z$. $\checkmark$

세 성질 만족 $\Rightarrow$ 동치관계. □

“순서”를 수학적으로 정의하자

우리가 일상에서 쓰는 “순서”

- $1 \leq 2 \leq 3 \leq \dots$ (실수 크기 비교)
- $\{1\} \subseteq \{1, 2\} \subseteq \{1, 2, 3\}$ (집합 포함)
- $2 \mid 6 \mid 12$ (나누어 떨어짐)
- $\text{Task A} \rightarrow \text{Task B} \rightarrow \text{Task C}$ (의존성)

공통점

모두 반사적, 반대칭적, 추이적 성질을 가진다!

⇒ 이 세 성질을 가진 관계를 **Partial Ordering (부분 순서)**라 부른다.

≤ 라는 기호에 대하여

새 기호 등장: $\preceq$

우리는 순서를 일반적으로 나타낼 때 $\preceq$ (소문자 preceq)라는 기호를 사용한다.

왜 새 기호를 쓰는가?

- $\leq$ 는 숫자의 크기 비교에만 쓰이는 기호로 굳어져 있다.
- 그런데 집합 포함 $\subseteq$, 나눗셈 $|$, 의존성 $\rightarrow$ 등 다양한 “순서”가 있다.
- 이 모든 순서를 한꺼번에 다루고 싶을 때 추상적인 기호 $\preceq$ 를 쓴다.

읽는 법

$a \preceq b$ 는 “a는 b보다 앞선다 (precedes) 혹은 같다”라고 읽는다.

$\preceq$ 는 $\leq$ 의 “일반화 버전”이라고 생각하면 된다.

≼의 다양한 얼굴

같은 ≼, 다른 해석

맥락	≼가 실제로 의미하는 것	예시
실수	$\leq$ (작거나 같다)	$3 \leq 5$
집합	$\subseteq$ (부분집합)	$\{1\} \subseteq \{1, 2\}$
양의 정수	$ $ (나누어 떨어짐)	$2 \leq 6$
문자열	prefix 관계	<code>"abc" ≼ "abcdef"</code>
작업	의존성 “먼저 해야 함”	<code>compile ≼ link</code>

핵심

≼ 자체가 의미를 갖는 게 아니라,
“어떤 집합 위에 어떤 순서관계를 얹었느냐”에 따라 ≼의 의미가 달라진다.

Partial Ordering 정의

정의 (Partial Order)

집합 A 위의 관계 R 이 다음 세 가지를 모두 만족하면 **부분순서 (partial ordering)**라 한다:

- 1 **Reflexive**: 모든 $a \in A$ 에 대해 $a R a$
- 2 **Antisymmetric**: $a R b$ 이고 $b R a$ 이면 $a = b$
- 3 **Transitive**: $a R b$ 이고 $b R c$ 이면 $a R c$

Poset

부분순서 R 이 얹혀진 집합 A 를 **Partially Ordered Set (Poset)**이라 부르고 (A, R) 또는 $(A, \preceq)$ 로 쓴다.

기억하자

반사 + 반대칭 + 추이 = 순서. Symmetric이 아닌 **Antisymmetric!**

Poset 예시 확인

예 1. $(\mathbb{Z}, \leq)$

- Reflexive: $a \leq a$. ✓
- Antisym: $a \leq b$ 이고 $b \leq a$ 이면 $a = b$. ✓
- Transitive: $a \leq b \leq c \Rightarrow a \leq c$. ✓

$\Rightarrow (\mathbb{Z}, \leq)$ 는 poset.

예 2. $(\mathcal{P}(S), \subseteq)$ — 멱집합 위의 포함관계

- $A \subseteq A$. ✓
- $A \subseteq B, B \subseteq A \Rightarrow A = B$. ✓
- $A \subseteq B \subseteq C \Rightarrow A \subseteq C$. ✓

예 3. $(\mathbb{Z}^+, |)$ — 양의 정수 나눗셈

- $a | a$. ✓
- $a | b, b | a \Rightarrow a = b$ (양의 정수 한정). ✓
- $a | b | c \Rightarrow a | c$. ✓

왜 “Partial”인가?

Comparable vs Incomparable

poset $(A, \preceq)$ 에서 두 원소 a, b 에 대해:

- $a \preceq b$ 또는 $b \preceq a$ 이면 “**비교 가능 (comparable)**”
- 둘 다 아니면 “**비교 불가능 (incomparable)**”

예시: $(\mathbb{Z}^+, |)$ 에서

- $2 \mid 6$ 이므로 2와 6은 comparable.
- 그런데 2와 3은? $2 \nmid 3$ 이고 $3 \nmid 2$. **incomparable!**

“Partial”의 의미

모든 쌍이 비교되지 않을 수 있어서 “partial (부분적)”이라 부른다.

모든 쌍이 비교 가능한 경우 $\Rightarrow$ **Total Order (전순서)** 또는 **Chain**.

예: $(\mathbb{Z}, \leq)$ 는 total. $(\mathbb{Z}^+, |)$ 는 partial만 됨.

Hasse Diagram이란?

동기

poset의 관계를 directed graph로 그리면, 화살표가 너무 많아서 복잡하다.
(reflexive, transitive 때문에 필요 이상의 간선이 생김)

꼭 필요한 간선만 남겨서 깔끔하게 그린 그림 $\Rightarrow$ Hasse diagram.

만드는 절차 (총 4단계)

- 1 poset의 관계를 directed graph로 그린다. (모든 (a, b) with $a \preceq b$)
- 2 **Self-loop 제거** (reflexive는 당연하므로)
- 3 **Transitive로 얻어지는 간선 제거** ($a \preceq b \preceq c$ 에서 $a \rightarrow c$ 삭제)
- 4 화살표 방향을 아래 $\rightarrow$ 위로 통일하고 화살표 머리를 생략

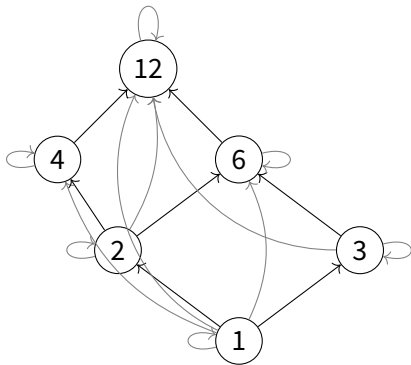
규칙

a 위에 b가 바로 그려져 있으면 $a \preceq b$. 실제 간선만 읽으면 된다.

Hasse Diagram 예시: $(\{1, 2, 3, 4, 6, 12\}, |)$

Step 1: 모든 (a, b) with $a \mid b$ 를 간선으로 그린다.

$1 \mid 1, 1 \mid 2, 1 \mid 3, 1 \mid 4, 1 \mid 6, 1 \mid 12, 2 \mid 2, 2 \mid 4, 2 \mid 6, 2 \mid 12, \dots$

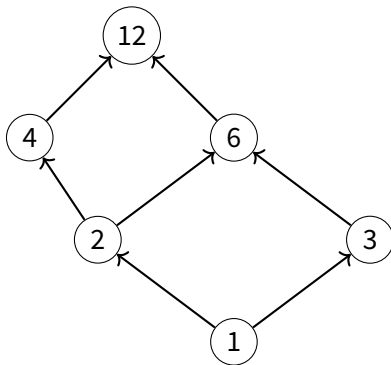


간선이 너무 많아서 복잡하다!

Hasse Diagram 만들기 — Step 2, 3

Step 2: Self-loop 제거

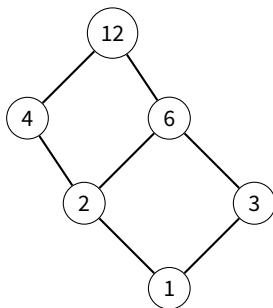
Step 3: Transitive 간선 제거 — $1 \rightarrow 4$ 는 $1 \rightarrow 2 \rightarrow 4$ 로 유도되므로 삭제. 마찬가지로 $1 \rightarrow 6$, $1 \rightarrow 12$, $2 \rightarrow 12$, $3 \rightarrow 12$ 삭제.



훨씬 깔끔해졌다. 남은 간선은 “바로 아래에서 바로 위”만.

Hasse Diagram 완성 — Step 4

Step 4: 아래→위 방향으로 통일, 화살표 머리 제거



읽는 법

- 2 위에 4, 6이 있다 $\Rightarrow 2 \mid 4, 2 \mid 6$
- 1 위에 (여러 단계를 통해) 12가 있다 $\Rightarrow 1 \mid 12$
- 2와 3은 서로 간의 경로가 없다 $\Rightarrow$ incomparable

Maximal / Minimal Element

직관

Hasse 그림에서 “맨 위” = **maximal**, “맨 아래” = **minimal**.
 (“나보다 위가 없음” = maximal, “나보다 아래가 없음” = minimal)

정의

$(A, \preceq)$ poset에서:

- a가 **maximal**: $a \preceq b$ 이고 $a \neq b$ 인 b가 없음
- a가 **minimal**: $b \preceq a$ 이고 $a \neq b$ 인 b가 없음

예시: $(\{1, 2, 3, 4, 6, 12\}, |)$

Maximal: 12 / Minimal: 1

주의

maximal/minimal은 여러 개일 수 있다! 예: $(\{2, 3\}, |)$ 은 둘 다 maximal이자 minimal.

Greatest / Least Element

직관

maximal보다 더 강한 조건:

- **Greatest**: 모든 원소보다 크다 (모두와 비교됨)
- **Maximal**: 자기보다 큰 게 없다 (incomparable 원소는 있어도 OK)

정의

- **a가 greatest**: 모든 $x \in A$ 에 대해 $x \preceq a$
- **a가 least**: 모든 $x \in A$ 에 대해 $a \preceq x$

예시 비교

- $(\{1, 2, 3, 4, 6, 12\}, |)$: **greatest** = 12, **least** = 1
- $(\{2, 3, 5, 7\}, |)$: **greatest/least 없음**. 모두 incomparable. maximal/minimal은 각각 네 개 모두.

Upper/Lower Bound, LUB, GLB

직관

poset의 부분집합 $B \subseteq A$ 에 대해:

- **Upper Bound:** B의 모든 원소보다 “위”에 있는 원소 (있을 수도 없을 수도)
- **Lower Bound:** B의 모든 원소보다 “아래”에 있는 원소
- **LUB (least upper bound) = Join $\vee$:** 모든 upper bound 중 가장 작은 것
- **GLB (greatest lower bound) = Meet $\wedge$:** 모든 lower bound 중 가장 큰 것

예시: $(\mathbb{Z}^+, |), B = \{4, 6\}$

- Upper bounds: 12, 24, 36, ... (4와 6 모두를 나누는 수로 나뉘는 수들)
- LUB = 12 = lcm(4, 6). **LUB가 최소공배수!**
- Lower bounds: 1, 2
- GLB = 2 = gcd(4, 6). **GLB가 최대공약수!**

Lattice — 직관적 이해

Lattice 직관

“어떤 두 원소든 만나는 점 (**meet**)과 잇는 점 (**join**)이 항상 존재하는 poset”

⇒ 두 원소를 집어들면 최대공약수 같은 것과 최소공배수 같은 것이 언제나 있는 구조

정의

poset $(A, \preceq)$ 에서 임의의 두 원소 a, b 에 대해 $\text{LUB}(\{a, b\})$ 와 $\text{GLB}(\{a, b\})$ 가 항상 존재하면 **lattice**.

CS에서 왜 중요?

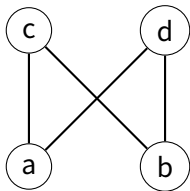
- 컴파일러 타입 체크: 공통 상위 타입 찾기 (LUB)
- 데이터베이스: 속성의 조합 (union/intersection)
- 불리언 대수: AND가 meet, OR가 join
- 버전 관리: 두 commit의 공통 조상 (merge base)

Lattice 예시와 비-예시

예시 1: $(\mathbb{Z}^+, |)$ 에서 $\text{join} = \text{lcm}$, $\text{meet} = \text{gcd} \Rightarrow \text{lattice} \checkmark$

예시 2: $(\mathcal{P}(S), \subseteq)$ 에서 $\text{join} = \cup$, $\text{meet} = \cap \Rightarrow \text{lattice} \checkmark$

비-예시: 아래 poset은 lattice **아님**.



- $\{c, d\}$ 의 upper bound: 없음! $\Rightarrow$ LUB도 없음 $\Rightarrow$ **lattice 아님**
- $\{a, b\}$ 의 lower bound: 없음. GLB도 없음.

Topological Sort — 왜 필요한가?

현실 문제

- **Build system:** `main.o`를 만들려면 `main.c`와 `util.h`가 먼저.
- `make`, `npm`, `cargo`는 파일들의 의존성을 보고 순서를 정한다.
- **강의 이수:** 이산수학은 선행과목이 있고, 알고리즘은 이산수학이 선행.
- **작업 스케줄:** A가 끝나야 B가 시작 가능 같은 의존성 그래프.

질문

“의존성을 위배하지 않는 전체 실행 순서를 하나 만들어줘.”

⇒ 수학적으로 이것은 **partial order**를 **total order**로 확장하는 것!

Topological Sort 정의

정의

poset $(A, \preceq)$ 의 **topological sort**란, A 의 원소를 일렬로 나열한 $a_1, a_2, \dots, a_n$ 인데, $a_i \preceq a_j$ 이면 반드시 $i \leq j$ (즉 순서대로)인 것.

직관

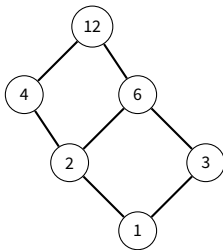
Hasse diagram을 보고 “아래에서 위로 차례대로 집어낼 수 있는 순서”를 만드는 것.

핵심 알고리즘: Kahn's algorithm (1962)

- 1 현재 poset에서 **minimal element** 하나 고른다.
- 2 그 원소를 출력하고 집합에서 제거한다.
- 3 집합이 빌 때까지 반복.

Topological Sort 예시 — 단계별

Poset $(\{1, 2, 3, 4, 6, 12\}, |)$ 의 Hasse:



Kahn 알고리즘 진행

- Step 1: $\min = 1 \rightarrow [1]$, 남은 $\{2, 3, 4, 6, 12\}$
- Step 2: $\min$ 후보 $\{2, 3\}$. 선택 2: $[1, 2]$, 남은 $\{3, 4, 6, 12\}$
- Step 3: $\min$ 후보 $\{3, 4\}$. 선택 3: $[1, 2, 3]$, 남은 $\{4, 6, 12\}$
- Step 4: $\min$ 후보 $\{4, 6\}$. 선택 4: $[1, 2, 3, 4]$, 남은 $\{6, 12\}$
- Step 5: $\min = 6 \rightarrow [1, 2, 3, 4, 6]$, 남은 $\{12\}$
- Step 6: $12 \rightarrow [1, 2, 3, 4, 6, 12] \checkmark$

Topological Sort — 여러 답이 가능

주의

Topological sort는 **일반적으로 유일하지 않다**.

선택 단계마다 여러 minimal이 있을 수 있으므로 답이 여러 개.

앞 예시의 또 다른 답

[1, 3, 2, 6, 4, 12]도 올바른 topological sort.

[1, 2, 4, 3, 6, 12]도 올바른 topological sort.

Partial order가 total order로

원래 관계에서는 2와 3이 incomparable이었지만,
topological sort한 결과(예: [1, 2, 3, 4, 6, 12])에서는 2가 3보다 앞에 있다.

⇒ 임의의 전체 순서를 부여했다 ⇒ partial → total로 “확장”했다.

함수 (Functions)

특별한 종류의 관계

앞으로 다룰 내용

- 함수는 사실 relation의 특수한 형태이다
- 정의역, 공역, 치역
- 일대일 (injection), 위로 (surjection), 일대일 대응 (bijection)
- 함수의 합성, 역함수

함수 = 특별한 관계 (직관)

고등학교에서 배운 함수

$f: A \rightarrow B$ 는 “A의 각 원소 a 에 B의 원소 $f(a)$ 를 하나씩 대응시키는 규칙”

Relation 관점에서 다시 보기

$f: A \rightarrow B$ 를 “ $A \times B$ 의 부분집합” (즉 A에서 B로의 관계)으로 보면:

$$f = \{(a, f(a)) : a \in A\}$$

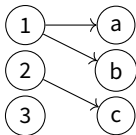
⇒ 함수는 다음 두 조건을 만족하는 관계:

- ① (존재성, **Totality**) 모든 $a \in A$ 에 대해 $(a, b) \in f$ 인 b 가 존재
- ② (유일성, **Well-defined**) 그 b 는 유일

한마디로

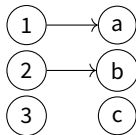
“입력이 있으면 출력이 반드시 있고, 그 출력은 딱 하나다.”

Relation vs Function 그림 비교



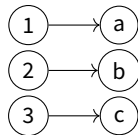
함수 아님

1이 두 개로 감 (유일성 X)



함수 아님

3에서 나가는 게 없음 (존재성 X)



함수 O

모든 입력 → 정확히 1개 출력

Domain, Codomain, Range

세 가지 집합을 구분하자

$f : A \rightarrow B$ 에서:

- **Domain (정의역)** = A : 입력들의 집합
- **Codomain (공역)** = B : 출력이 “놓일 수 있는” 집합
- **Range (치역 / Image)** = $f(A) = \{f(a) : a \in A\}$: 실제로 출력된 값들

예시: $f : \mathbb{Z} \rightarrow \mathbb{Z}, f(x) = x^2$

- Domain: $\mathbb{Z}$
- Codomain: $\mathbb{Z}$
- Range: $\{0, 1, 4, 9, 16, \dots\}$ (제곱수들만) $\subsetneq \mathbb{Z}$

중요

Range $\subseteq$ Codomain. 같을 수도, 아닐 수도.
같은 때가 “surjection”이다 (곧 배움).

Injection (일대일 함수) — 직관

직관

“서로 다른 입력은 서로 다른 출력으로 간다.”

⇒ 충돌 (collision) 없음. CS에서는 **perfect hash**라 부른다.

정의

$f: A \rightarrow B$ 가 **injective** (**one-to-one, 1:1**) 일 조건:

$$a_1 \neq a_2 \Rightarrow f(a_1) \neq f(a_2)$$

(대우로 더 쓰기 쉽다:) $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$

판별 팁

두 원소의 f 값이 같다고 가정했을 때,
그 두 원소도 같다는 것을 유도할 수 있으면 injective.

Injection 예시

예 1. $f: \mathbb{Z} \rightarrow \mathbb{Z}, f(x) = 2x + 1$

$f(a) = f(b) \Rightarrow 2a + 1 = 2b + 1 \Rightarrow a = b. \checkmark$

$\Rightarrow$ **Injective.**

예 2. $g: \mathbb{Z} \rightarrow \mathbb{Z}, g(x) = x^2$

$g(2) = 4$ 이고 $g(-2) = 4$. $2 \neq -2$ 지만 함수값 같음.

$\Rightarrow$ **Not injective.**

CS 관점

- **해시 함수:** injective면 이상적 (collision 없음)
- **ID 발급:** 사용자 $\rightarrow$ user_id는 injective여야 함
- **암호화:** 복호화 가능하려면 injective (사실은 bijective)

Surjection (위로의 함수) — 직관

직관

“공역의 모든 원소가 적어도 한 번은 맞혀진다.”

⇒ $\text{Range} = \text{Codomain}$. CS적으로: 모든 결과 값이 도달 가능.

정의

$f: A \rightarrow B$ 가 **surjective (onto)**일 조건:

모든 $b \in B$ 에 대해, 어떤 $a \in A$ 가 존재하여 $f(a) = b$.

판별 팁

임의의 $b \in B$ 를 잡고, $f(a) = b$ 를 만족하는 a 를 구성할 수 있는지 보면 된다.

Surjection 예시

예 1. $f: \mathbb{Z} \rightarrow \mathbb{Z}, f(x) = x + 1$

임의 b 에 대해 $a = b - 1$ 이면 $f(a) = b$. ✓

⇒ **Surjective.**

예 2. $g: \mathbb{Z} \rightarrow \mathbb{Z}, g(x) = x^2$

g 의 치역은 $\{0, 1, 4, 9, \dots\}$. 음수는 안 맞힘.

$b = -1$ 은 도달 불가.

⇒ **Not surjective.**

예 3. $h: \mathbb{Z} \rightarrow \{0, 1, \dots, 9\}, h(x) = x \bmod 10$

모든 나머지 $0, 1, \dots, 9$ 가능 ⇒ **Surjective.**

하지만 injective 아님 ($h(3) = h(13) = 3$).

Bijection (일대일 대응) — 직관

Bijection = Injection + Surjection

“입력과 출력이 정확히 짝을 이룬다.”

A의 모든 원소가 B의 유일한 원소와 매칭되고, 빠지는 게 없다.

CS에서 왜 중요?

- 역함수가 존재하려면 반드시 bijection이어야 함.
- 암호화: 평문과 암호문이 완벽히 짝이 되어야 복호화 가능.
- 순열/permutation: $\{1, \dots, n\}$ 에서 $\{1, \dots, n\}$ 으로 가는 bijection.
- 인코딩/디코딩: 정보 손실 없는 변환은 항상 bijection.

예시

$f: \mathbb{Z} \rightarrow \mathbb{Z}, f(x) = x + 3 \Rightarrow \text{bijection.}$

$g: \{1, 2, 3\} \rightarrow \{a, b, c\}, g(1) = b, g(2) = a, g(3) = c \Rightarrow \text{bijection.}$

네 가지 경우 정리표

	Surjective O	Surjective X
Injective O	Bijection (완벽 대응) $f(x) = x + 1$	Injection only $f : \mathbb{N} \rightarrow \mathbb{N}, f(x) = 2x$
Injective X	Surjection only $h(x) = x \bmod 10$	Neither $g(x) = x^2$ on $\mathbb{Z}$

기억법

- **Injection:** “서로 다르게 매핑” ($\neq \rightarrow \neq$)
- **Surjection:** “모두 맞춤” (Range = Codomain)
- **Bijection:** 둘 다

함수의 합성 (Composition)

직관

“먼저 f 로 보내고, 결과를 다시 g 로 보낸다.”

Pipeline, unix |, 함수형 프로그래밍의 . 연산자 같은 것.

정의

$f : A \rightarrow B, g : B \rightarrow C$ 일 때 합성 $g \circ f : A \rightarrow C$ 는

$(g \circ f)(a) = g(f(a))$ 로 정의. f 의 codomain이 g 의 domain이어야 함.

예시

$f(x) = x + 1, g(x) = x^2$ 이면

$$(g \circ f)(x) = g(x + 1) = (x + 1)^2 = x^2 + 2x + 1$$

$$(f \circ g)(x) = f(x^2) = x^2 + 1$$

순서가 중요! $g \circ f \neq f \circ g$

역함수 (Inverse Function)

직관

함수가 하는 일을 “되돌리는” 함수.

f 가 a 를 b 로 보냈다면, f^{-1} 은 b 를 다시 a 로 돌려보냄.

존재 조건

$f : A \rightarrow B$ 의 역함수가 존재하려면 f 가 **bijection**이어야 함!

- Injective 아니면 $\rightarrow$ 한 b 에 여러 a 가 대응 $\rightarrow$ 유일성 위반
- Surjective 아니면 $\rightarrow$ 어떤 b 는 돌려보낼 a 가 없음 $\rightarrow$ 존재성 위반

정의 및 예시

f 가 bijection일 때 $f^{-1} : B \rightarrow A$ 는 $f^{-1}(b) = a \iff f(a) = b$.

$f(x) = x + 3$ 이면 $f^{-1}(x) = x - 3$.

$f \circ f^{-1} = f^{-1} \circ f = \text{id}$ (항등함수).

대표적인 함수들

Floor와 Ceiling

- $\lfloor x \rfloor$ (floor) = x 이하의 최대 정수. 예: $\lfloor 3.7 \rfloor = 3$, $\lfloor -1.2 \rfloor = -2$
- $\lceil x \rceil$ (ceiling) = x 이상의 최소 정수. 예: $\lceil 3.2 \rceil = 4$
- CS 활용: 배열 인덱싱, 페이지 수 계산, 이진 탐색 중간점

모듈로 함수

$f(x) = x \bmod m$ — 해싱, 암호학에서 핵심.

항등함수, 상수함수

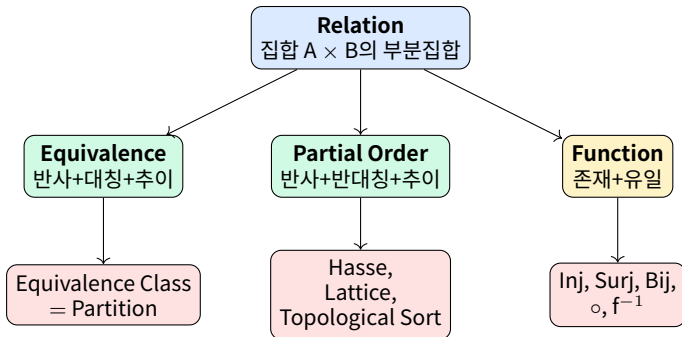
$\text{id}(x) = x$, $f(x) = c$ (상수)

연결

Floor와 ceiling은 **injective** 아님, **surjective** (to $\mathbb{Z}$).

항등함수는 **bijection**. 상수함수는 (정의역 크기 > 1 일 때) 둘 다 아님.

Big Picture: 이번 주에 배운 것



핵심

세 개의 “관계 종류”가 서로 다른 CS 구조를 낳는다:

동치관계 → **분할** (Union-Find, 해시 버킷, 커넥티드 컴포넌트)

부분순서 → **위상 정렬** (빌드, 의존성)

함수 → **매핑** (해시, 암호, 변환)

핵심 알고리즘 복습

Boolean Matrix 기반

- **Boolean product** $M_R \odot M_S$: 관계 합성, $O(n^3)$
- **Warshall's algorithm**: Transitive closure R^* 계산, $O(n^3)$

Graph 기반

- **DFS/BFS**: connectivity, 동치클래스 탐색
- **Kahn's Topological Sort**: $O(V + E)$
- **Union-Find (DSU)**: 동치관계의 동적 관리, 거의 $O(\alpha(n))$

다음 주 예고

Week 4: Counting (조합론)

순열, 조합, 이항계수, 비둘기집 원리, 포함-배제 원리.

오늘 배운 집합/함수/bijection이 다시 등장한다! (예: $n!$ 가 bijection 개수)

마무리 — 기억해야 할 5가지

- ➊ **Relation**의 세 표현법: **집합 / Boolean Matrix / Directed Graph**. 상황에 맞게 골라 쓴다.
- ➋ **5가지 성질**: Reflexive, Irreflexive, Symmetric, Antisymmetric, Transitive. 각 성질이 matrix와 graph에서 어떻게 보이는지 기억!
- ➌ **Warshall's algorithm**은 transitive closure를 $O(n^3)$ 에 계산. Boolean matrix와 graph 관점 모두 이해하자.
- ➍ **동치관계** $\leftrightarrow$ **분할**. 이 대응은 CS 자료구조 (Union-Find, 해시)의 수학적 기반.
- ➎ **함수 = 특별한 관계** (존재 + 유일). Injective, Surjective, Bijective의 차이와 역함수 존재 조건.

수고하셨습니다! 질문 환영합니다.